Experiment No: 06

Experiment Name: Student Information System (SIS)

1. Introduction

Student Information System (SIS) is a software system used to manage student-related information such as student profiles, academic results processing, GPA calculation, and teacher evaluation.

In traditional systems, student records and result processing are handled manually, which often leads to errors, data inconsistency, and delays. This system automates the process to ensure **data integrity, accuracy, and efficiency**.

The main focus of this system is maintaining **accurate student data and reliable GPA calculation**, along with proper evaluation of teachers.

2. Objectives

- To automate student information management
- To maintain student profiles efficiently
- To process exam results accurately
- To calculate GPA automatically
- To manage teacher evaluation system
- To ensure data integrity and consistency

3. Functional Requirements

The system should be able to:

- Register new students
- Allow student login and profile management
- Store and update student academic records
- Input and process exam results
- Calculate GPA automatically

- Generate grade reports
- Allow teachers to submit marks
- Evaluate teachers based on student feedback
- Maintain history of student performance
- Generate reports for administration

4. Non-Functional Requirements

The system should:

- Provide fast and efficient performance
- Ensure data integrity and accuracy
- Be secure and protect sensitive data
- Be user-friendly and easy to use
- Support backup and recovery
- Be reliable and scalable

5. Use Case Modeling

Actors:

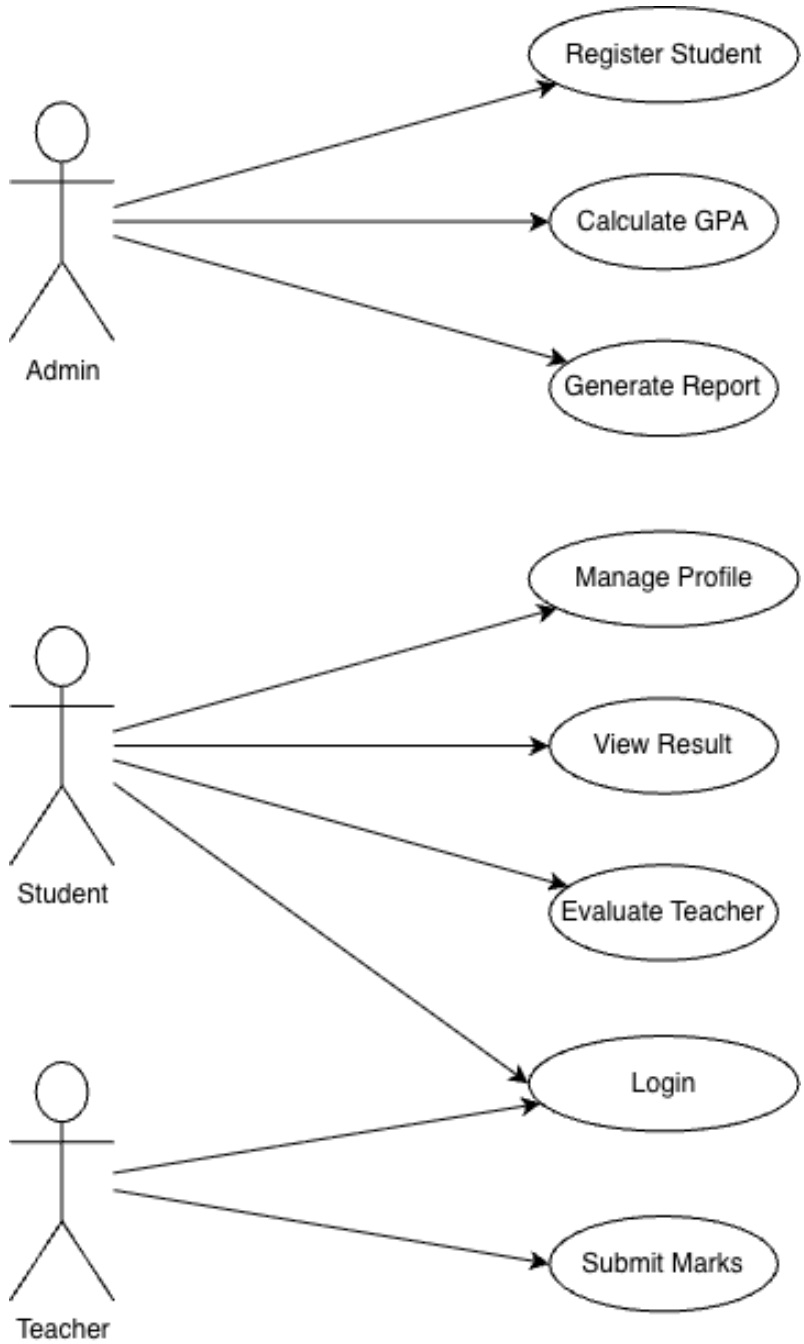
- Student
- Teacher
- Admin

Main Use Cases:

- Register Student
- Login
- Manage Profile
- Submit Marks
- Calculate GPA
- View Result

- Evaluate Teacher
- Generate Report

Use Case Diagram:



6. Structural Design (Class Diagram)

Main Classes:

- User
- Student
- Teacher
- Course
- Result
- GPA
- Evaluation

Description:

- User is the parent class
- Student and Teacher inherit from User
- Student enrolls in courses
- Teacher teaches courses and submits marks
- Result stores marks and grades
- GPA calculates student performance
- Evaluation stores teacher ratings

Attributes and Methods

User

- Attributes: id, name
- Method: login()

Student

- Method: viewResult(), giveEvaluation()

Teacher

- Method: submitMarks()

Course

- Attributes: course_id, course_name

Result

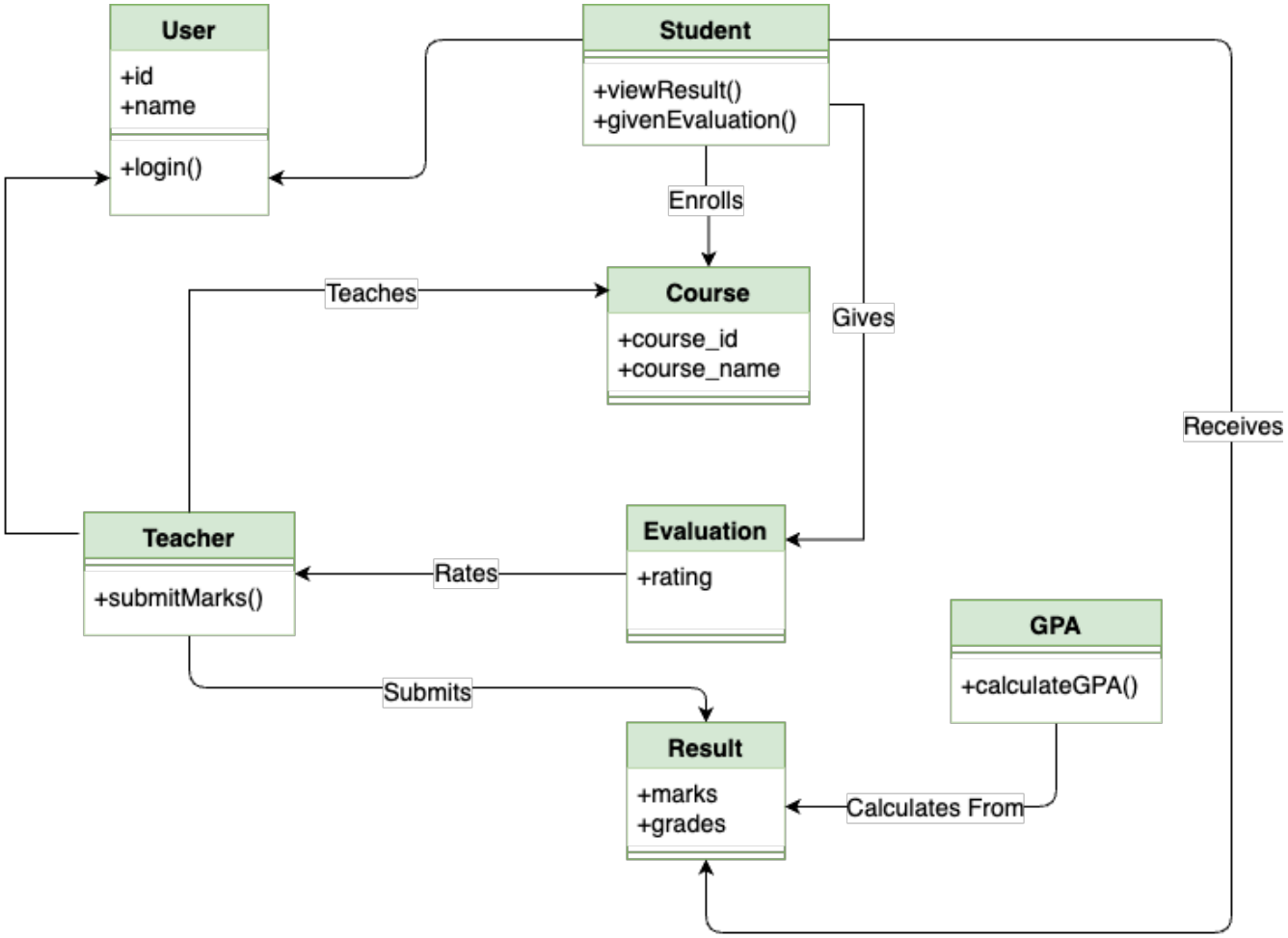
- Attributes: marks, grade

GPA

- Method: calculateGPA()

Evaluation

- Attribute: rating



7. Database Design (ER Diagram)

7.1 Entities

- Student
- Teacher
- Course
- Result
- Evaluation

7.2 Relationships

- Student enrolls in Course
- Teacher teaches Course
- Teacher submits Result
- Student receives Result
- Student gives Evaluation to Teacher

7.3 Suggested Primary Keys and Foreign Keys

STUDENT

- student_id (PK)
- name
- department

TEACHER

- teacher_id (PK)
- name
- department

COURSE

- course_id (PK)
- course_name

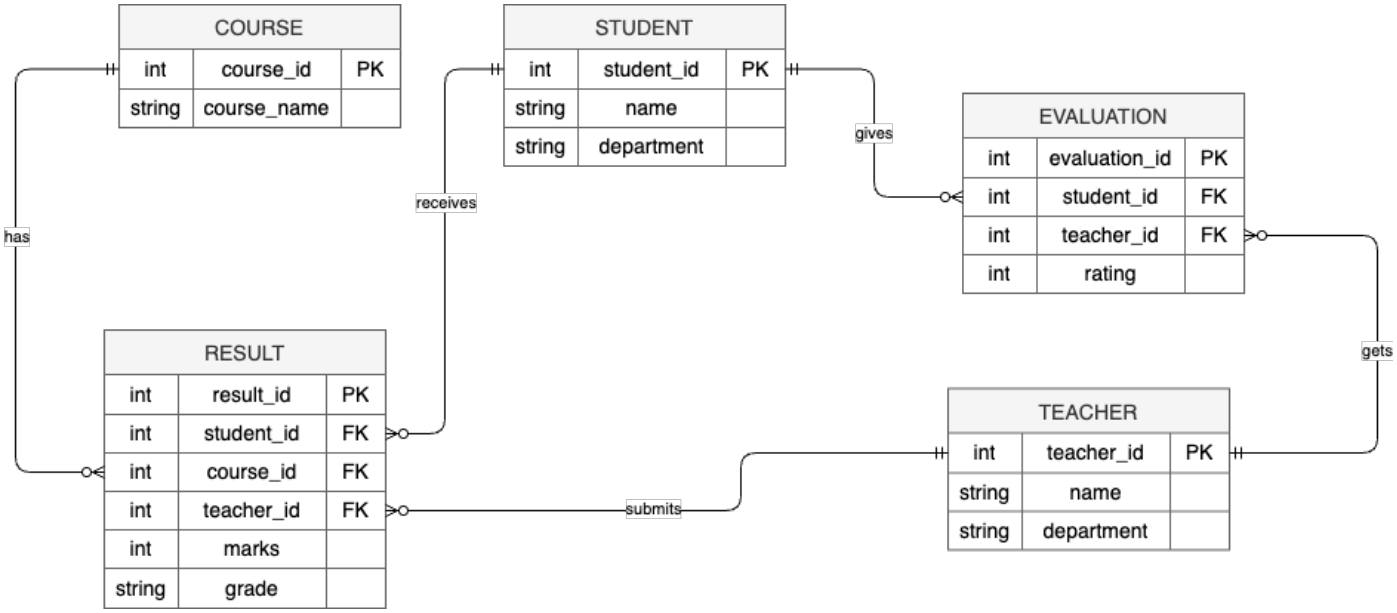
RESULT

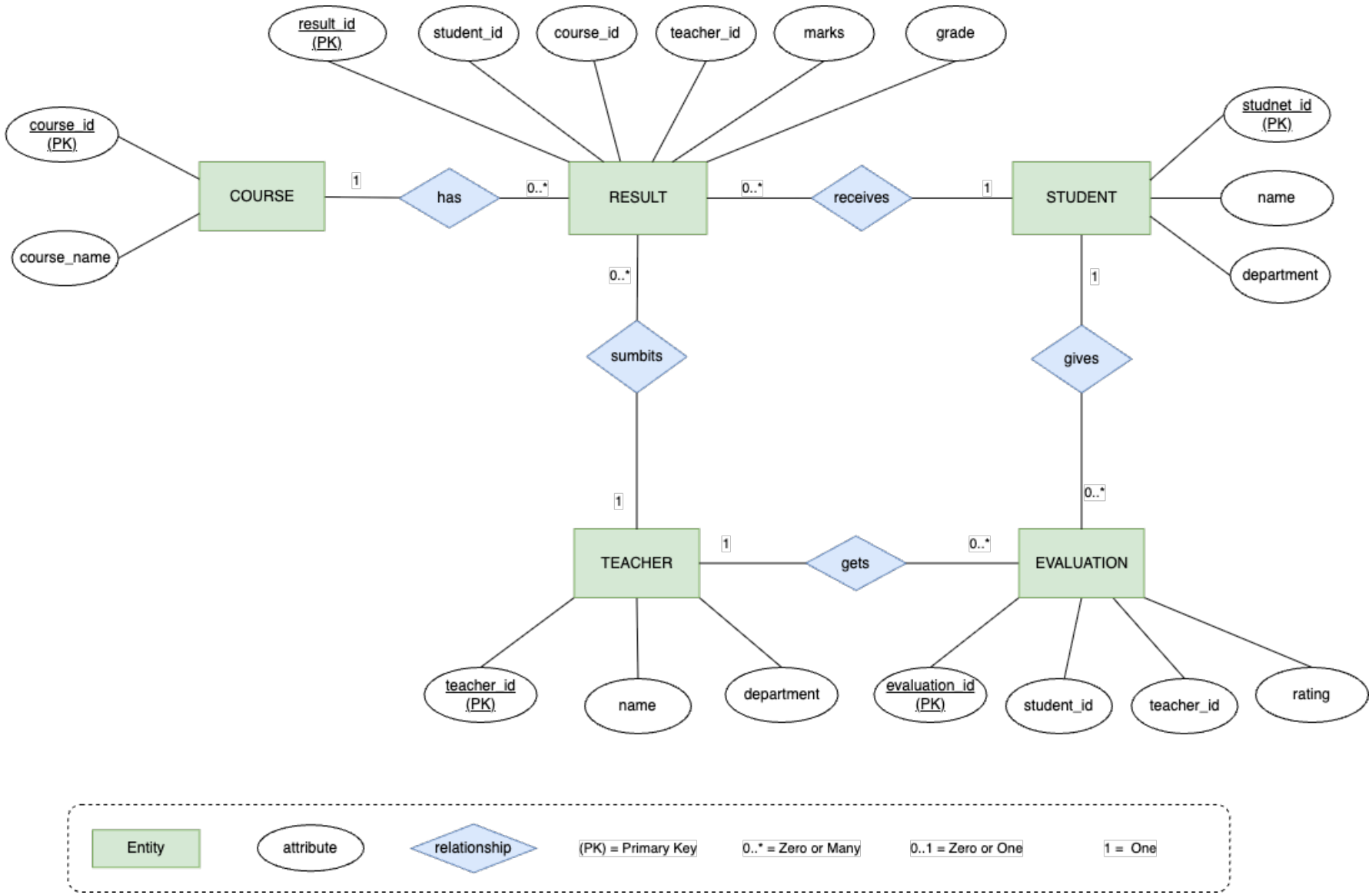
- result_id (PK)

- student_id (FK)
- course_id (FK)
- marks
- grade

EVALUATION

- evaluation_id (PK)
- student_id (FK)
- teacher_id (FK)
- rating




result_id (PK)
student_id
course_id
teacher_id
marks
grade
course_id (PK)
course_name
COURSE
1
has
0..*
RESULT
0..*
receives
1
STUDENT
studnet_id (PK)
name
department
0..*
sumbits
1
gives
0..*
1
TEACHER
1
gets
0..*
EVALUATION
teacher_id (PK)
name
department
evaluation_id (PK)
student_id
teacher_id
rating
Entity
attribute
relationship
(PK) = Primary Key
0..* = Zero or Many
0..1 = Zero or One
1 = One

8. Advantages

- Reduces manual errors
- Ensures accurate GPA calculation
- Maintains data integrity
- Saves time and effort
- Easy access to student information
- Improves decision-making

9. Limitations

- Requires technical knowledge
- Initial setup cost is high
- System failure may affect operations
- Needs proper security measures

10. Conclusion

The Student Information System (SIS) is an effective solution for managing student data digitally. It ensures accurate result processing, reliable GPA calculation, and proper teacher evaluation. The system improves efficiency, reduces errors, and maintains strong data integrity, making it essential for modern educational institutions.

11. Simple Web Implementation

index.html

```html
<!DOCTYPE html>
<html>
<head>
  <title>Student Information System</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
```

```html
<header class="hero">
  <h1>Student Information System</h1>
  <p>Offline Portal with GPA Calculation</p>
</header>

<nav>
  <a href="#student">Student</a>
  <a href="#courses">Courses</a>
  <a href="#marksSection">Marks</a>
  <a href="#evaluation">Evaluation</a>
  <a href="#reports">Reports</a>
</nav>

<div class="container">

<section id="student" class="card">
  <h2>Register Student</h2>
  <form id="studentForm">
    <input type="text" id="sname" placeholder="Student Name" required>
    <input type="text" id="sid" placeholder="Student ID" required>
    <input type="text" id="dept" placeholder="Department" required>
    <button type="submit">Register Student</button>
  </form>
</section>

<section id="courses" class="card">
  <h2>Course List</h2>
  <div id="courseList"></div>
</section>

<section id="marksSection" class="card">
  <h2>Submit Marks</h2>
  <form id="marksForm">
    <input type="text" id="markStudentId" placeholder="Student ID" required>
    <input type="text" id="courseId" placeholder="Course ID" required>
    <input type="number" id="marks" placeholder="Marks" required>
    <button type="submit">Submit Marks</button>
  </form>
</section>

<section id="evaluation" class="card">
  <h2>Teacher Evaluation</h2>
  <form id="evaluationForm">
    <input type="text" id="evalStudentId" placeholder="Student ID" required>
    <input type="text" id="teacherName" placeholder="Teacher Name" required>
    <input type="number" id="rating" placeholder="Rating 1-5" min="1" max="5"
required>
```

```html
    <button type="submit">Submit Evaluation</button>
  </form>
</section>

<section id="reports" class="card">
  <div class="section-title">
    <h2>Student Reports</h2>
    <button onclick="clearReports()" class="danger-btn">Clear Reports</button>
  </div>

  <div id="reportList"></div>
</section>
</div>

<script src="script.js"></script>
</body>
</html>
```

styles.css

```css
body {
  font-family: Arial;
  background: #f4f7fb;
  margin: 0;
}

.hero {
  background: linear-gradient(135deg, #1e3a8a, #2563eb);
  color: white;
  text-align: center;
  padding: 25px;
}

nav {
  background: white;
  padding: 12px;
  text-align: center;
  box-shadow: 0 3px 10px rgba(0,0,0,0.08);
}

nav a {
  margin: 10px;
  text-decoration: none;
  color: #1d4ed8;
  font-weight: bold;
}
```

```css
.container {
  width: 85%;
  margin: auto;
}

.card {
  background: white;
  padding: 20px;
  margin: 20px 0;
  border-radius: 12px;
  box-shadow: 0 8px 20px rgba(0,0,0,0.08);
}

input {
  display: block;
  margin: 10px 0;
  padding: 11px;
  width: 100%;
  box-sizing: border-box;
}

button,
.upload-btn {
  padding: 10px 14px;
  background: #2563eb;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  display: inline-block;
}

button:hover,
.upload-btn:hover {
  background: #1d4ed8;
}

.report-card {
  border-left: 5px solid #2563eb;
}

.good {
  color: green;
  font-weight: bold;
}

.average {
```

```
  color: orange;
  font-weight: bold;
}

.fail {
  color: red;
  font-weight: bold;
}

.section-title {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.danger-btn {
  background: #dc2626;
}

.danger-btn:hover {
  background: #b91c1c;
}

.backup-buttons {
  margin-bottom: 15px;
}
```

script.js

```
const courseList = document.getElementById("courseList");
const reportList = document.getElementById("reportList");

let courses = [];

function getStudents() {
  return JSON.parse(localStorage.getItem("students")) || [];
}

function saveStudents(students) {
  localStorage.setItem("students", JSON.stringify(students));
}

function getReports() {
  return JSON.parse(localStorage.getItem("studentReports")) || [];
}
```

```javascript
function saveReports(reports) {
  localStorage.setItem("studentReports", JSON.stringify(reports));
}

// Load courses from courses.txt
async function loadCourses() {
  try {
    const res = await fetch("courses.txt");
    const text = await res.text();

    courses = text.trim().split("\n").map(line => {
      const [id, name, credit] = line.split("|");

      return {
        id: id.trim(),
        name: name.trim(),
        credit: Number(credit.trim())
      };
    });

    renderCourses();
  } catch {
    courseList.innerHTML = "<p>Could not load courses.txt</p>";
  }
}

function renderCourses() {
  courseList.innerHTML = courses.map(course => `
    <div class="card">
      <p><b>Course ID:</b> ${course.id}</p>
      <p><b>Course Name:</b> ${course.name}</p>
      <p><b>Credit:</b> ${course.credit}</p>
    </div>
  `).join("");
}

// Register student
document.getElementById("studentForm").addEventListener("submit", function(e) {
  e.preventDefault();

  const name = document.getElementById("sname").value.trim();
  const id = document.getElementById("sid").value.trim();
  const dept = document.getElementById("dept").value.trim();

  let students = getStudents();

  if (students.find(s => s.id === id)) {
```

```
    alert("Student already registered.");
    return;
  }

  students.push({ id, name, dept });
  saveStudents(students);

  alert("Student registered successfully.");
  this.reset();
});

// Grade calculation
function getGradePoint(marks) {
  if (marks >= 80) return { grade: "A+", point: 4.00 };
  if (marks >= 75) return { grade: "A", point: 3.75 };
  if (marks >= 70) return { grade: "A-", point: 3.50 };
  if (marks >= 65) return { grade: "B+", point: 3.25 };
  if (marks >= 60) return { grade: "B", point: 3.00 };
  if (marks >= 55) return { grade: "B-", point: 2.75 };
  if (marks >= 50) return { grade: "C+", point: 2.50 };
  if (marks >= 45) return { grade: "C", point: 2.25 };
  if (marks >= 40) return { grade: "D", point: 2.00 };
  return { grade: "F", point: 0.00 };
}

// Submit marks
document.getElementById("marksForm").addEventListener("submit", function(e) {
  e.preventDefault();

  const studentId = document.getElementById("markStudentId").value.trim();
  const courseId = document.getElementById("courseId").value.trim();
  const marksInput = document.getElementById("marks").value;
  const marks = Number(marksInput);

  if (marksInput === "" || isNaN(marks) || marks < 0 || marks > 100) {
    alert("Please enter valid marks between 0 and 100.");
    return;
  }

  const students = getStudents();
  const student = students.find(s => s.id === studentId);
  const course = courses.find(c => c.id === courseId);

  if (!student) {
    alert("Student not found.");
    return;
  }
```

```
  if (!course) {
    alert("Course not found.");
    return;
  }

  const result = getGradePoint(marks);

  let reports = getReports();

  // Remove previous result for same student and same course
  reports = reports.filter(r => !(r.studentId === studentId && r.courseId ===
courseId));

  reports.push({
    studentId: student.id,
    studentName: student.name,
    department: student.dept,
    courseId: course.id,
    courseName: course.name,
    credit: Number(course.credit),
    marks: marks,
    grade: result.grade,
    point: Number(result.point)
  });

  saveReports(reports);
  renderReports();

  alert("Marks submitted successfully.");
  this.reset();
});

// GPA calculation
function calculateGPA(studentId) {
  const reports = getReports().filter(r => r.studentId === studentId);

  let totalCredit = 0;
  let totalWeightedPoint = 0;

  reports.forEach(r => {
    const credit = Number(r.credit);
    const point = Number(r.point);

    totalCredit += credit;
    totalWeightedPoint += credit * point;
  });
```

```
  if (totalCredit === 0) return "0.00";

  return (totalWeightedPoint / totalCredit).toFixed(2);
}

// Render reports
function renderReports() {
  const reports = getReports();

  if (reports.length === 0) {
    reportList.innerHTML = "<p>No reports available.</p>";
    return;
  }

  reportList.innerHTML = reports.map(r => {
    const gpa = calculateGPA(r.studentId);

    return `
      <div class="card report-card">
        <p><b>${r.studentName}</b> (${r.studentId})</p>
        <p><b>Department:</b> ${r.department}</p>
        <p><b>Course:</b> ${r.courseName}</p>
        <p><b>Marks:</b> ${r.marks}</p>
        <p><b>Grade:</b> ${r.grade}</p>
        <p><b>Point:</b> ${r.point}</p>
        <p><b>GPA:</b> ${gpa}</p>
      </div>
    `;
  }).join("");
}

// Clear reports
function clearReports() {
  localStorage.removeItem("studentReports");
  renderReports();
}

loadCourses();
renderReports();
```

courses.txt

```
ICT101|Introduction to Programming|3

ICT102|Data Structures|3
```

```
ICT201|Database Management System|3
ICT202|Software Engineering|3
ICT301|Computer Networks|3
ICT302|Operating System|3
ICT4106|System Analysis Lab|1.5
```

reports.txt

```
2018040|Rahim Ahmed|ICT|ICT101|Introduction to Programming|3|85|A+|4|4.00
2018040|Rahim Ahmed|ICT|ICT4106|System Analysis Lab|1.5|78|A|3.75|3.92
```